

Práctica: Comparación de Plantillas (Template Matching)

La comparación de plantillas consiste en buscar dentro de una imagen grande una pequeña imagen o patrón llamado “plantilla”.

Por ejemplo:

- buscar un tornillo dentro de una fotografía;
- encontrar un barreno (agujero) en un engrane;
- encontrar a Waldo dentro de una imagen llena de personas.

OpenCV realiza esto con la función:

```
cv2.matchTemplate()
```

La idea es simple:

1. Se toma la plantilla.
2. Se desliza sobre toda la imagen.
3. En cada posición se calcula qué tan parecida es.
4. La posición con mayor similitud es la encontrada.

Introducción a la comparación de plantillas

Supongamos que tenemos:

- `imagen.jpg` : imagen grande
- `plantilla.jpg` : objeto pequeño que queremos encontrar

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread('imagen.jpg', cv2.IMREAD_GRAYSCALE)
template = cv2.imread('plantilla.jpg', cv2.IMREAD_GRAYSCALE)

h, w = template.shape

resultado = cv2.matchTemplate(img, template, cv2.TM_CCOEFF_NORMED)

min_val, max_val, min_loc, max_loc = cv2.minMaxLoc(resultado)

top_left = max_loc
bottom_right = (top_left[0] + w, top_left[1] + h)

img_color = cv2.cvtColor(img, cv2.COLOR_GRAY2RGB)

cv2.rectangle(img_color, top_left, bottom_right, (255,0,0), 2)

plt.figure(figsize=(10,6))
plt.imshow(img_color)
plt.title('Plantilla encontrada')
```

```
plt.axis('off')
plt.show()
```

Explicación

```
resultado = cv2.matchTemplate(img, template, cv2.TM_CCOEFF_NORMED)
```

genera una matriz donde cada valor indica qué tan parecida es la plantilla en esa posición.

Luego:

```
max_loc
```

indica la posición donde la similitud es mayor.

Búsqueda de un barreno en un engrane

Supongamos que se tiene:

- una imagen de un engrane completo;
- una pequeña imagen de un barreno circular.

La meta es encontrar dónde está ese barreno.

```
engranaje = cv2.imread('engrane.jpg', cv2.IMREAD_GRAYSCALE)
barreno = cv2.imread('barreno.jpg', cv2.IMREAD_GRAYSCALE)

h, w = barreno.shape

resultado = cv2.matchTemplate(
    engranaje,
    barreno,
    cv2.TM_CCOEFF_NORMED
)

_, max_val, _, max_loc = cv2.minMaxLoc(resultado)

print("Similitud:", max_val)

img_color = cv2.cvtColor(engranaje, cv2.COLOR_GRAY2RGB)

cv2.rectangle(
    img_color,
    max_loc,
    (max_loc[0]+w, max_loc[1]+h),
    (255,0,0),
    3
)

plt.figure(figsize=(8,8))
plt.imshow(img_color)
plt.title('Barreno encontrado')
```

```
plt.axis('off')
plt.show()
```

Preguntas

- ¿Qué sucede si la plantilla es más grande que el barreno?
- ¿Qué ocurre si el barreno está rotado?

Limitación importante

La comparación de plantillas funciona bien solo si:

- el tamaño es parecido;
- la orientación es la misma;
- la iluminación no cambia mucho.

Buscando a Waldo

La clásica actividad de “¿Dónde está Waldo?” es un excelente ejemplo de template matching.

Se tiene:

- Una imagen grande con muchas personas;
- Una pequeña imagen del rostro o cuerpo de Waldo.

```
img = cv2.imread('waldo_grande.jpg', cv2.IMREAD_GRAYSCALE)
waldo = cv2.imread('waldo_plantilla.jpg', cv2.IMREAD_GRAYSCALE)

h, w = waldo.shape

resultado = cv2.matchTemplate(
    img,
    waldo,
    cv2.TM_CCOEFF_NORMED
)

_, max_val, _, max_loc = cv2.minMaxLoc(resultado)

print("Confianza:", max_val)

img_color = cv2.cvtColor(img, cv2.COLOR_GRAY2RGB)

cv2.rectangle(
    img_color,
    max_loc,
    (max_loc[0]+w, max_loc[1]+h),
    (255,0,0),
    3
)

plt.figure(figsize=(12,8))
plt.imshow(img_color)
```

```
plt.title('Waldo encontrado')
plt.axis('off')
plt.show()
```

Actividad

- Probar con otras plantillas:
 - sombrero;
 - gafas;
 - personaje diferente.
- Comparar qué plantilla encuentra mejor a Waldo.

Detección de múltiples barrenos en un engrane

A diferencia del primer caso, ahora queremos detectar todos los barrenos, no solo uno.

Para ello se usa un umbral de similitud.

```
engranaje = cv2.imread('engrane.jpg', cv2.IMREAD_GRAYSCALE)
barreno = cv2.imread('barreno.jpg', cv2.IMREAD_GRAYSCALE)

h, w = barreno.shape

resultado = cv2.matchTemplate(
    engranaje,
    barreno,
    cv2.TM_CCOEFF_NORMED
)

umbral = 0.75

ubicaciones = zip(*((resultado >= umbral).nonzero()[::-1]))

img_color = cv2.cvtColor(engranaje, cv2.COLOR_GRAY2RGB)

contador = 0

for punto in ubicaciones:
    contador += 1

    cv2.rectangle(
        img_color,
        punto,
        (punto[0]+w, punto[1]+h),
        (255,0,0),
        2
    )

plt.figure(figsize=(10,10))
plt.imshow(img_color)
plt.title(f'Barrenos encontrados: {contador}')
plt.axis('off')
plt.show()
```

Explicación

```
resultado >= umbral
```

selecciona todas las posiciones donde la similitud es suficientemente alta.

Por ejemplo:

- umbral = 0.95 : solo coincidencias casi perfectas;
- umbral = 0.70 : aparecen más coincidencias, pero también falsos positivos.

Actividad:

- Probar distintos umbrales.
- Determinar cuál produce el mejor resultado.

Comparación entre encontrar uno o varios objetos

Caso	Método
Buscar un solo objeto	<code>cv2.minMaxLoc()</code>
Buscar muchos objetos	<code>Umbral + nonzero()</code>

Aplicación real

La comparación de plantillas puede usarse en:

- Detección de piezas faltantes en industria
- Identificación de logos
- Reconocimiento de símbolos
- Control de calidad
- Detección de defectos repetitivos

Por ejemplo, una empresa puede usar template matching para verificar si todos los agujeros de una pieza metálica están presentes.